



Microsoft Developer Experience GitHub Copilot Demo

08/14/2025

Gary.Ciampa@microsoft.com

Azure Solution Engineer



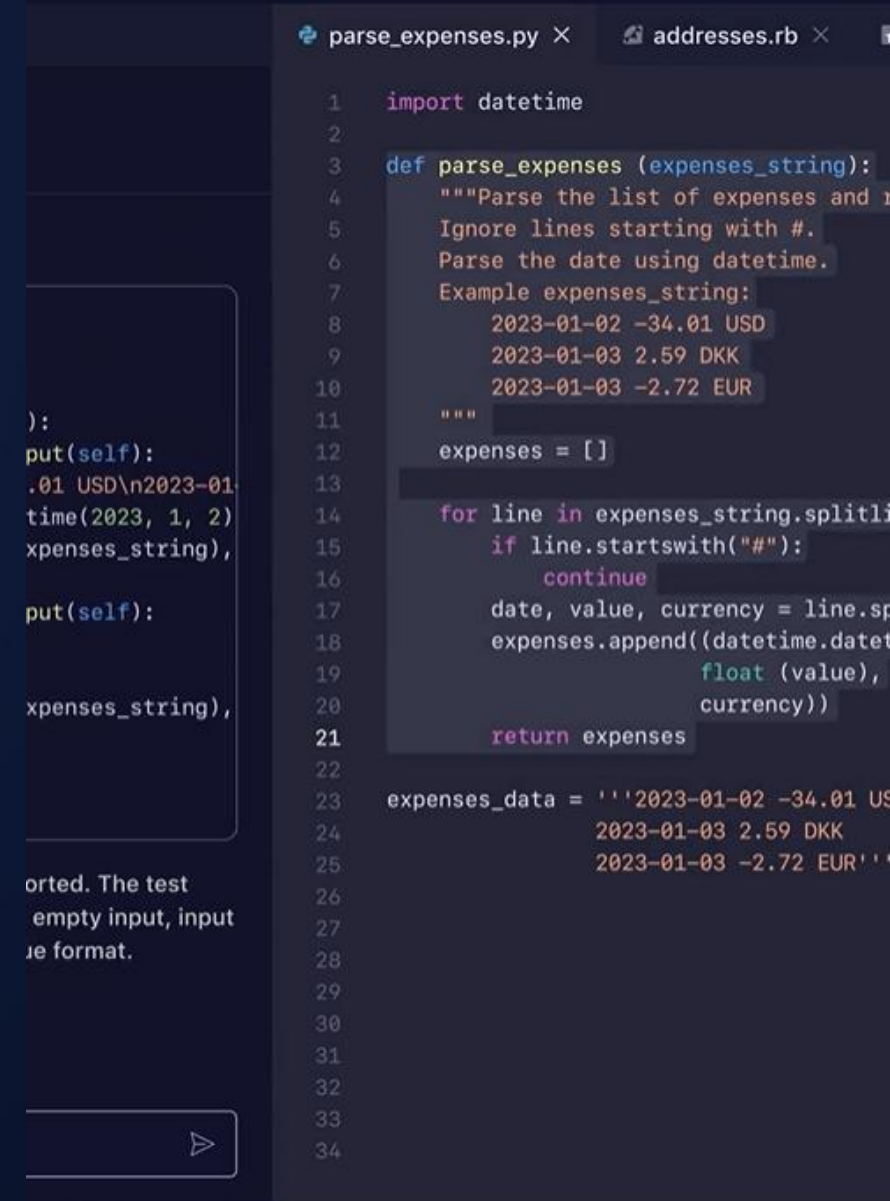
GitHub Copilot



Visual Studio Code

AGENDA

- Prolog:
 - Overview
 - Prompt Engineering
 - Developer copilot interaction
 - Data flows & patterns
 - Azure Functions scenario review
- Demo: GitHub CoPilot scenario(s), usage
- References & Next Steps



The screenshot shows a code editor with two tabs: `parse_expenses.py` and `addresses.rb`. The `parse_expenses.py` tab is active, displaying a Python function `parse_expenses` that processes a string of expenses. The function includes a docstring with an example input string and a list of expenses. The code uses `datetime` for date parsing and `float` for value parsing. The `addresses.rb` tab is partially visible, showing a Ruby file. Below the code editor, there is a text area with the text: "orted. The test empty input, input ie format." and a button with a right-pointing triangle.

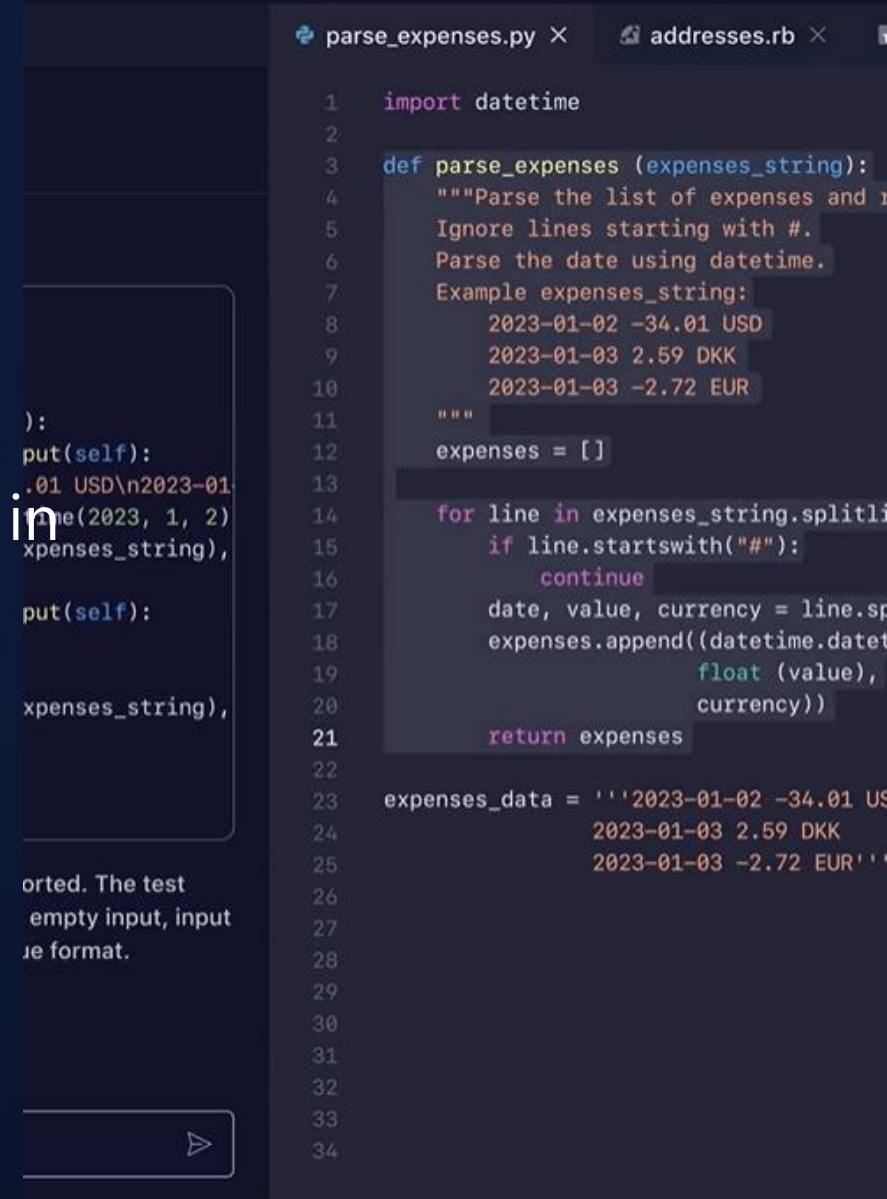
```
1 import datetime
2
3 def parse_expenses (expenses_string):
4     """Parse the list of expenses and
5     Ignore lines starting with #.
6     Parse the date using datetime.
7     Example expenses_string:
8         2023-01-02 -34.01 USD
9         2023-01-03 2.59 DKK
10        2023-01-03 -2.72 EUR
11    """
12    expenses = []
13
14    for line in expenses_string.splitlines():
15        if line.startswith("#"):
16            continue
17        date, value, currency = line.split()
18        expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),
19                        float(value),
20                        currency))
21    return expenses
22
23 expenses_data = '''2023-01-02 -34.01 USD
24                   2023-01-03 2.59 DKK
25                   2023-01-03 -2.72 EUR'''
26
27
28
29
30
31
32
33
34
```

orted. The test
empty input, input
ie format.



AI pair/peer-programmer

- AI developer tool, helps write code faster with less work
- Draws context from comments & code to suggest code/functions
- Developers code faster, focus on solving problems, stay in context & feel more fulfilled with their work
- Powered by OpenAI Codex, generative pretrained language models



The screenshot displays an AI pair programmer interface. On the left, a sidebar shows a list of code snippets with a search bar and a 'Run' button. The main area on the right is a code editor with two tabs: 'parse_expenses.py' and 'addresses.rb'. The 'parse_expenses.py' tab is active, showing a Python function 'def parse_expenses (expenses_string):' with a docstring and a list of example expenses. The function uses 'datetime' to parse dates and 'float' to parse values. The 'addresses.rb' tab is also visible, showing a Ruby function 'def parse_addresses (addresses_string):' with a docstring and a list of example addresses. The interface is dark-themed with blue and green accents.

```
def parse_expenses (expenses_string):  
    """Parse the list of expenses and return a list of datetime objects.  
    Ignore lines starting with #.  
    Parse the date using datetime.  
    Example expenses_string:  
        2023-01-02 -34.01 USD  
        2023-01-03 2.59 DKK  
        2023-01-03 -2.72 EUR  
    """  
    expenses = []  
    for line in expenses_string.splitlines():  
        if line.startswith("#"):  
            continue  
        date, value, currency = line.split()  
        expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),  
                        float(value),  
                        currency))  
    return expenses
```

```
expenses_data = '''2023-01-02 -34.01 USD  
2023-01-03 2.59 DKK  
2023-01-03 -2.72 EUR'''
```

Supported IDEs



Visual Studio



Visual Studio
Code



Vim/Neovim



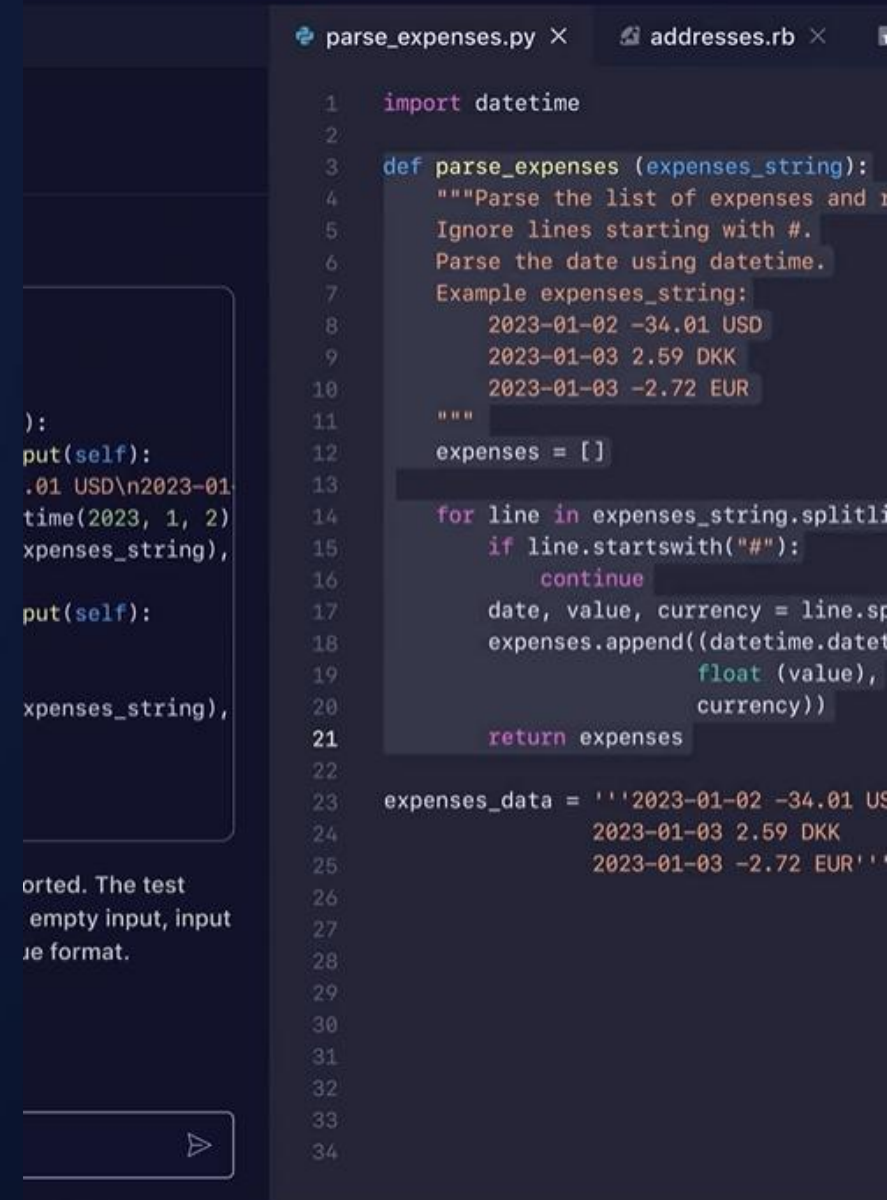
Jet Brains

GitHub CoPilot Emphasis Items

- CoPilot is exactly that, co-pilot, not a replacement
- Training data, hints & suggestions
 - IOW: sometimes, recommendations are wrong
- Predictive, based on semantics & context

Developer is PIC

- Design patterns & structure
- Coding standards, source management & style
- Context & snippets must be vetted for expected output



Limitations

“I’m powered by AI, so surprises and mistakes are possible. Make sure to verify any generated code or suggestions, and share feedback so that we can learn and improve.”

Copilot 

Copilot



GitHub CoPilot prompt engineering (4s')

- **Single:** focus prompt on a single, well-defined task or question. Clarity is crucial for eliciting accurate and useful responses.
- **Specific:** instructions are explicit and detailed. Specificity leads to more applicable and precise code suggestions.
- **Short:** keep prompts concise and to the point, balance ensures clarity without overloading.
- **Surround:** retain environment elements for tailored code suggestions, filenames, variables, methods

```
1 import datetime
2
3 def parse_expenses (expenses_string):
4     """Parse the list of expenses and
5     Ignore lines starting with #.
6     Parse the date using datetime.
7     Example expenses_string:
8         2023-01-02 -34.01 USD
9         2023-01-03 2.59 DKK
10        2023-01-03 -2.72 EUR
11    """
12    expenses = []
13
14    for line in expenses_string.splitlines():
15        if line.startswith("#"):
16            continue
17        date, value, currency = line.split()
18        expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),
19                        float(value),
20                        currency))
21    return expenses
22
23 expenses_data = '''2023-01-02 -34.01 USD
24                  2023-01-03 2.59 DKK
25                  2023-01-03 -2.72 EUR'''
26
27
28
29
30
31
32
33
34
```

Techniques



Zero-Shot Prompting

No Example



One-Shot Prompting

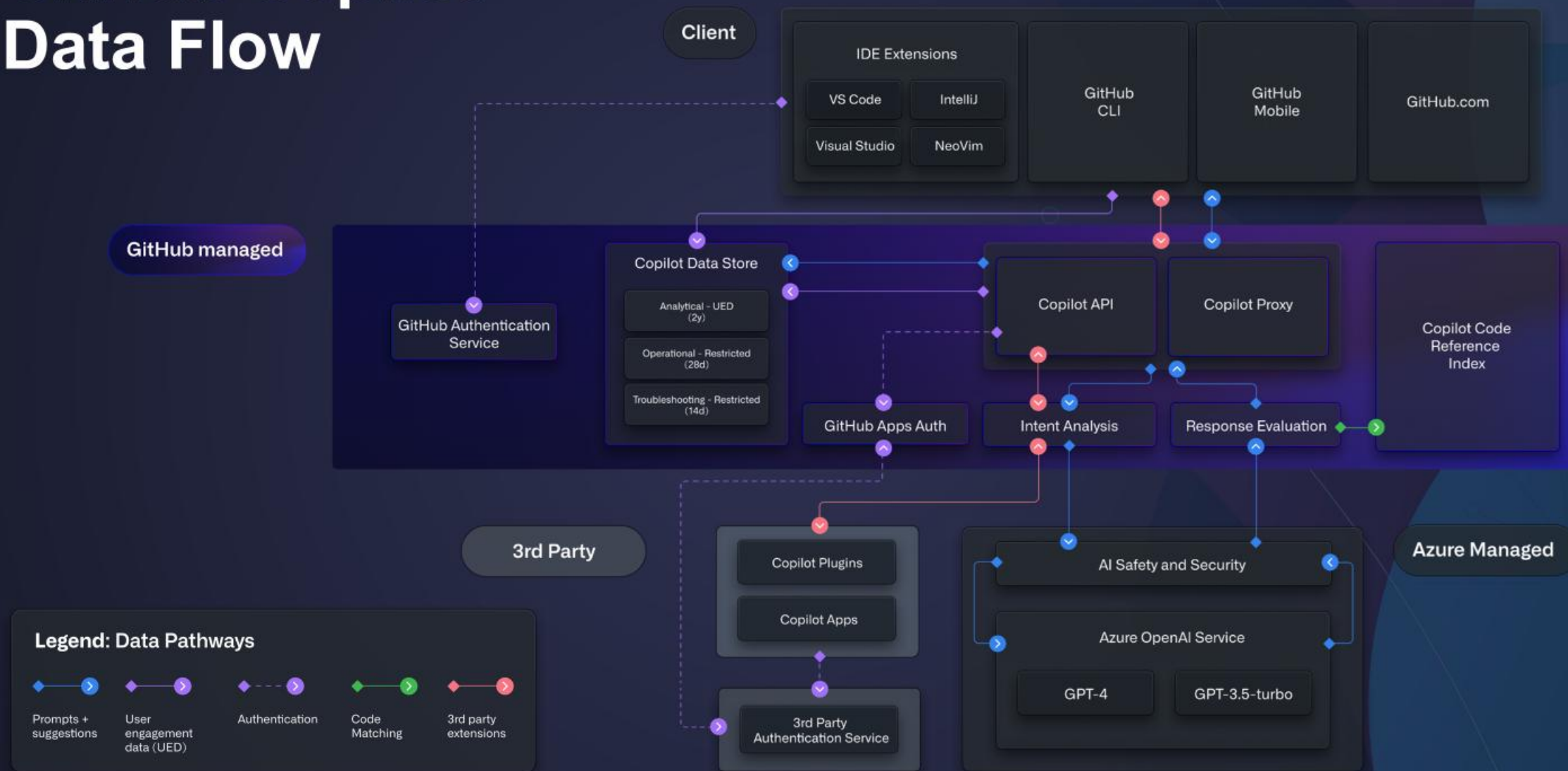
One Example



Few-Shot Prompting

Handful of Examples

GitHub Copilot Data Flow



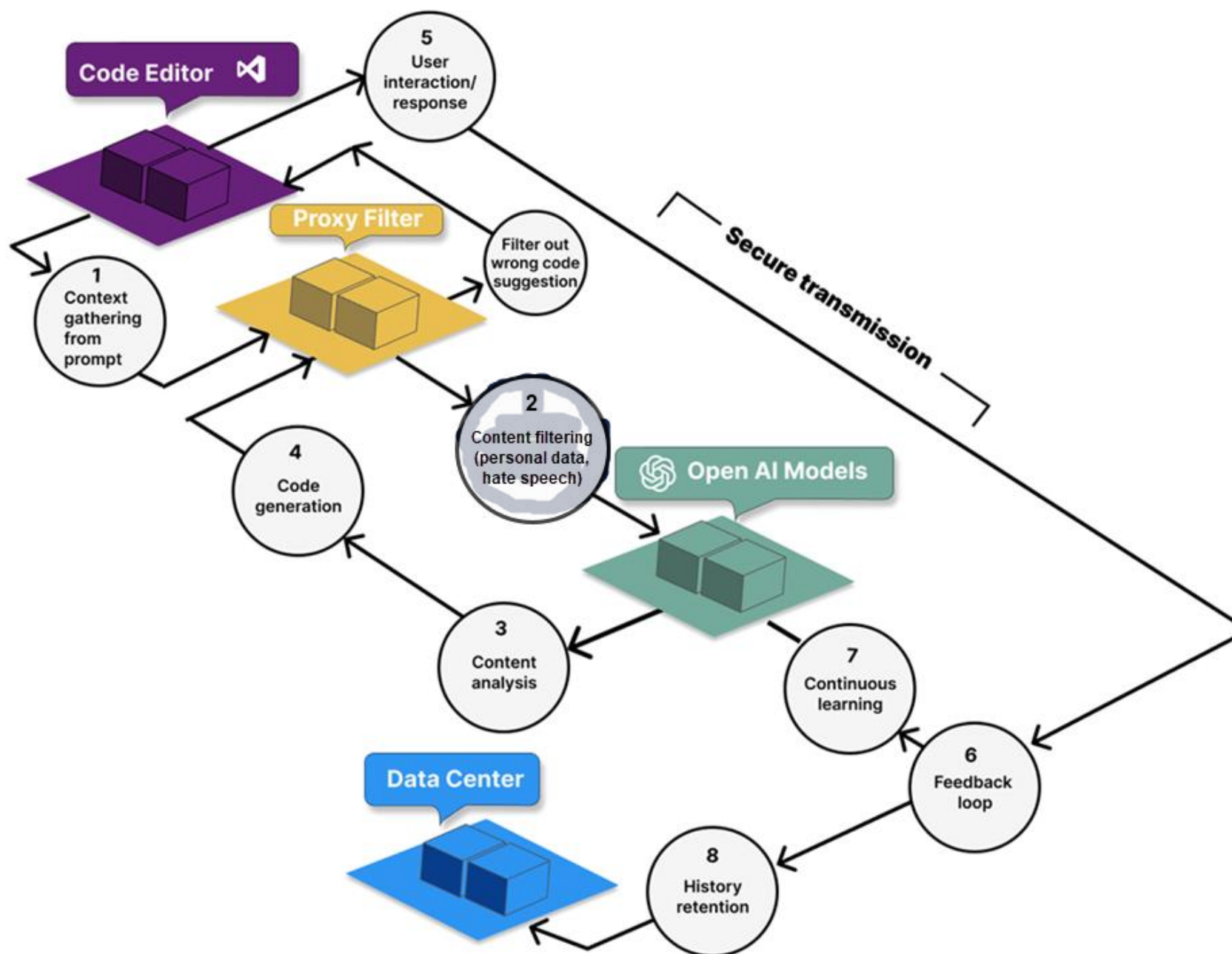


Illustration of Copilot Prompt Processing Flow

GitHub CoPilot references

[GitHub CoPilot landing page](#)

[GitHub CoPilot Trust Center](#)

[GitHub CoPilot LinkedIn Tips & Tricks](#)

[GitHub CoPilot for Skeptics, MAY 2025 BRK124](#)

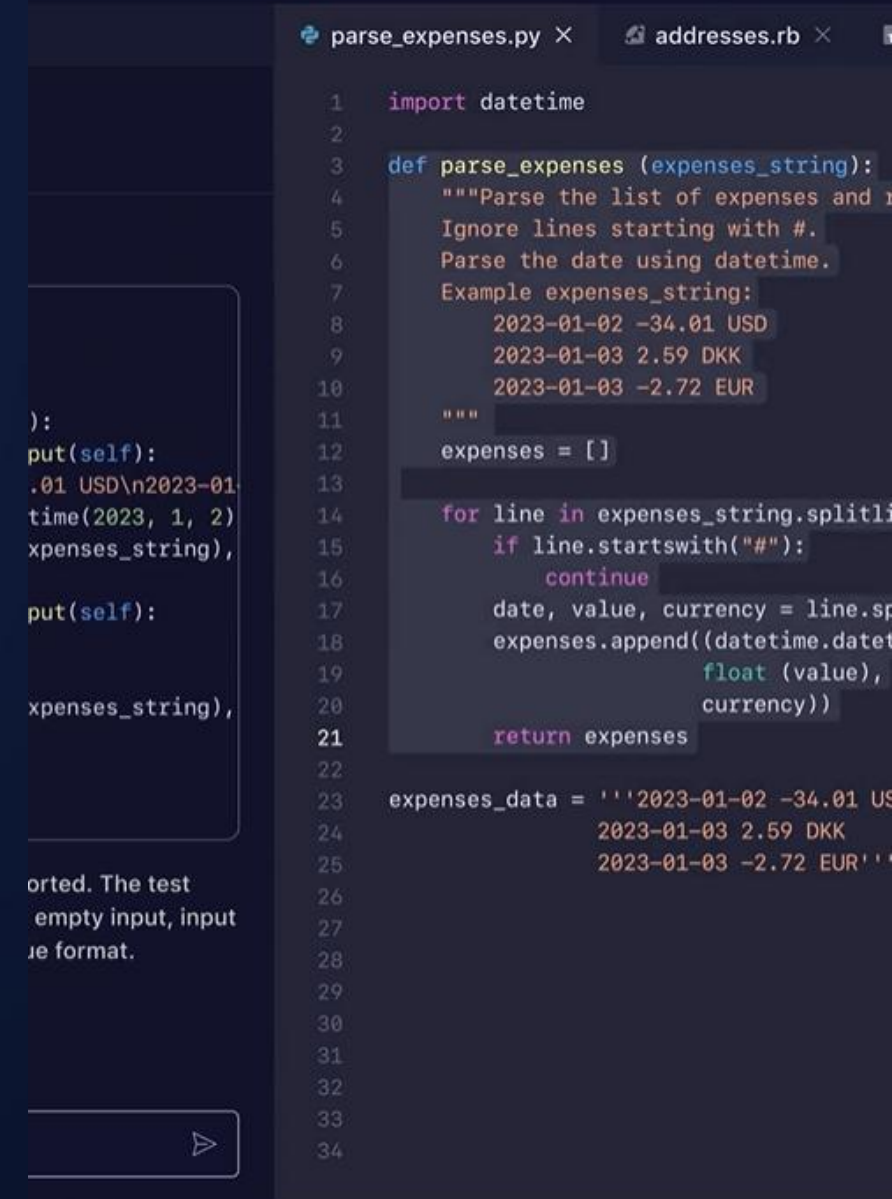
[GitHub CoPilot MSFT Learning Path](#)

[GitHub CoPilot YouTube Series](#)

[GitHub CoPilot Agent Mode 101](#)

[Awesome GitHub CoPilot Customizations](#)

[Visual Studio Code YouTube channel](#)





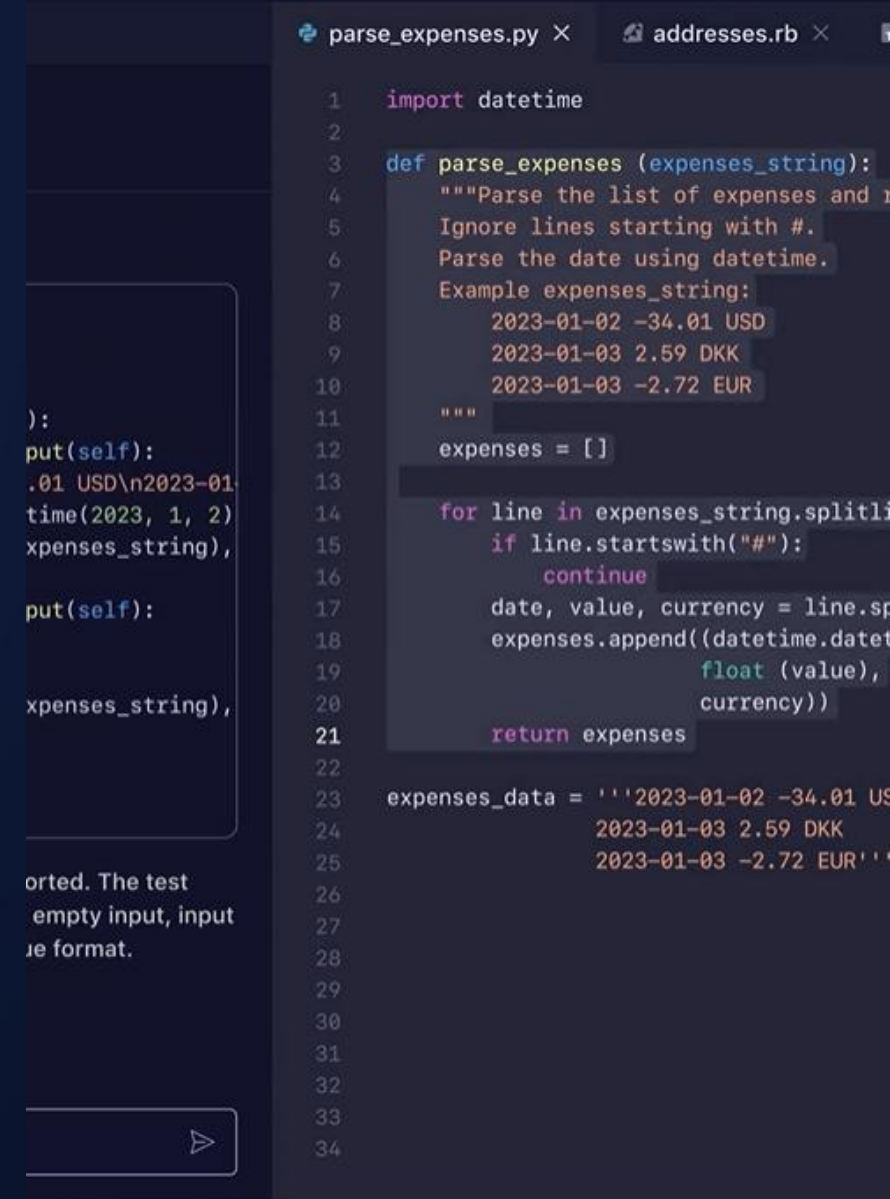
GitHub Copilot



Visual Studio Code

Exploring GitHub Copilot with Microsoft Fabric

- Integrate [Fabric Workspaces with a Git repository](#)
 - ✓ Clone the repository & work in Visual Studio Code
 - ✓ GitHub Copilot Scenario
 - ✓ Fabric references
 - ✓ Code changes & commit to Git repo
- Scenario
 - ✓ Developing or extending Fabric items (Data Warehouses, Notebooks, Semantic Models, KQL Databases, etc.)
 - ✓ Code migration
 - ✓ Code refactoring
 - ✓ Readme.md
- Reference scenarios
 - ✓ [Fabric data engineering, Spark notebooks](#)
 - ✓ [SQL database, Fabric data warehouses & Fabric SQL databases](#)
 - ✓ [Tabular model definition, for Fabric semantic models](#)



The screenshot shows a code editor with two tabs: 'parse_expenses.py' and 'addresses.rb'. The 'parse_expenses.py' tab is active, displaying Python code. The code defines a function 'parse_expenses' that takes 'expenses_string' as input. It includes a docstring explaining the function's purpose: to parse a list of expenses, ignoring lines starting with '#', and using 'datetime' for date parsing. An example of the input string is provided: '2023-01-02 -34.01 USD', '2023-01-03 2.59 DKK', and '2023-01-03 -2.72 EUR'. The function then processes this string, splitting it into lines, skipping comment lines, and parsing the remaining lines into a list of tuples containing date, value, and currency. The final output is a list of parsed expenses.

```
1 import datetime
2
3 def parse_expenses (expenses_string):
4     """Parse the list of expenses and
5     Ignore lines starting with #.
6     Parse the date using datetime.
7     Example expenses_string:
8         2023-01-02 -34.01 USD
9         2023-01-03 2.59 DKK
10        2023-01-03 -2.72 EUR
11    """
12    expenses = []
13
14    for line in expenses_string.splitlines():
15        if line.startswith("#"):
16            continue
17        date, value, currency = line.split()
18        expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),
19                        float(value),
20                        currency))
21    return expenses
22
23 expenses_data = '''2023-01-02 -34.01 USD
24                 2023-01-03 2.59 DKK
25                 2023-01-03 -2.72 EUR'''
26
27
28
29
30
31
32
33
34
```

GitHub Copilot at a glance

AI that builds with you

GitHub Copilot is the world’s most widely adopted AI developer tool and the competitive advantage developers ask for by name. It provides AI Assistance throughout the development lifecycle—within the IDE and GitHub.com—to streamline development during planning, coding, pull requests, and more.

Focus on tasks you love—leave the rest to agents

Automate repetitive work, streamline code reviews, and resolve issues with agent mode in integrated development environments (IDEs). Agent mode reasons through the problem, coordinates next steps, and applies the changes—while keeping you in the driver’s seat.

GitHub Copilot Business
\$19 user / month

Includes 300 premium requests per user month

GitHub Copilot Enterprise
\$39 user / month

Includes 1,000 premium requests per user per month

Unlimited use of base model
Additional premium requests at \$0.04 USD*

CAPABILITIES	DESCRIPTION
IDE Support	GitHub Copilot supports the most IDEs and integrated with GitHub.com. IDEs supported include Visual Studio Code, Visual Studio, JetBrains, IntelliJ, Eclipse, Neovim, and Xcode.
Inline Code Completions	New custom-trained GPT-4o model and advanced predictive edits will reduce latency.
Agentic AI Development	Agent Mode helps you perform sweeping changes quickly by analyzing code, proposing edits, running tests, and validating results across multiple files. Code review analyzes your work, uncovers hidden bugs, fixes mistakes, and more—before a human ever sees it.
Multi-Model	Supports latest models from OpenAI, Anthropic, and Gemini. See GitHub docs for model use cases.
Responsible AI, Certifications, Copyright/ Indemnity and Data Privacy	- Enterprise-level control to exclude code and public code license filtering/annotations. See GitHub Copilot Trust Center for details. - SOC 2 Type II, ISO/IEC 27001:2013 certifications and CSA STAR compliance. - Code generated by Copilot is your intellectual property and protected should a copyright issue arise.
Extensibility and Scalability	- Copilot’s capabilities can be extended and re-used using VS Code extension APIs - Support for Model Context Protocol (MCP) available - Single sign-on (SSO) in both Business and Enterprise

GitHub Copilot Scenario

1

Feature pulled from backlog for Sprint

2

Peer development team lacks required **time** & **experience** to implement feature

3

Director requested assistance to implement

4

Complete within 3 days & handoff to team for integration

Feature requirements

1

Develop a compute service endpoint in Azure?

2

Requires a secure method to store secrets?

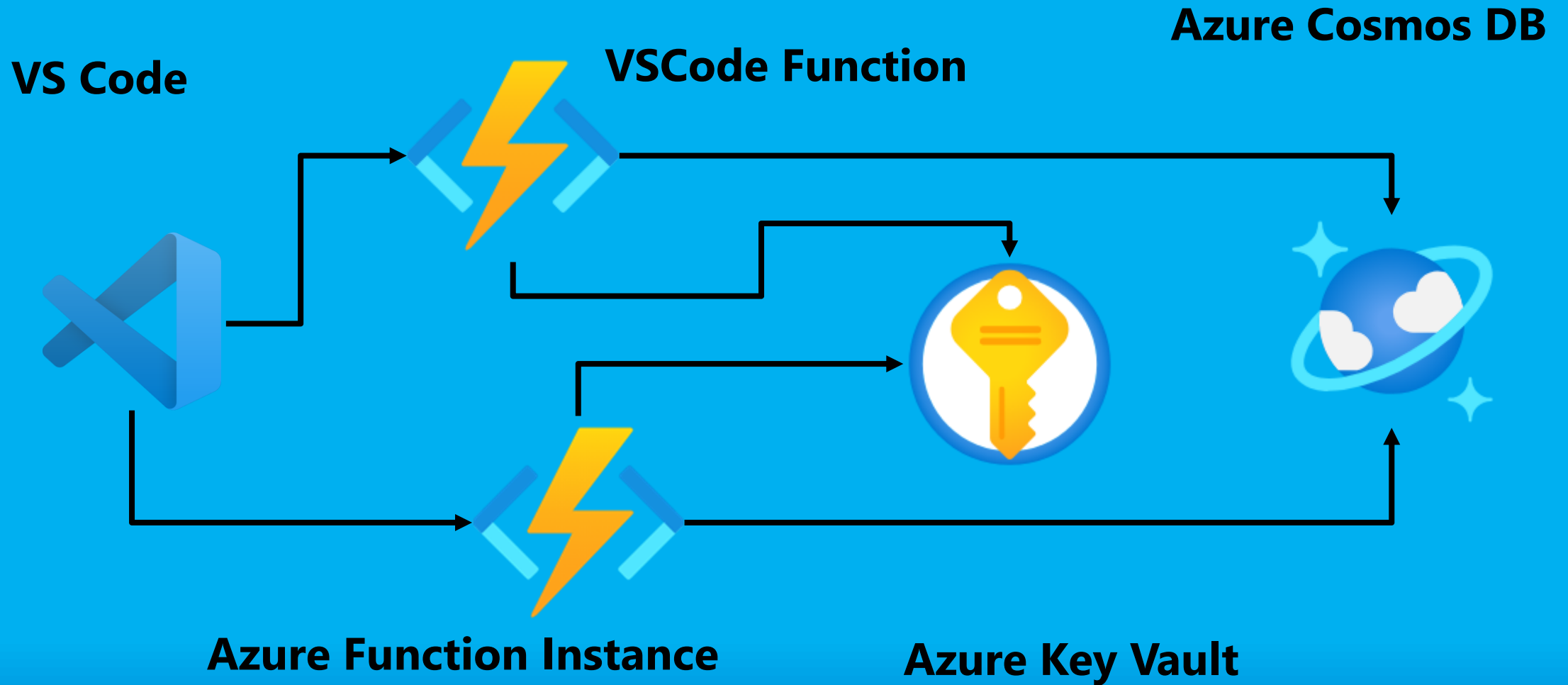
3

Persist data to Cosmos DB?

4

Service must be scalable to handle dynamic workloads?

Conceptual Architecture Diagram



Quick start questions, considerations

- Azure Service: [Azure compute decision tree](#)?
- Visual Studio Code Extensions?
- .NET packages required?
- Prototype, scaling, REST endpoint?
- Securely manage, access a secret?
- Store persistent data?
- Challenges
 - ✓ Learning curve & .NET language
 - ✓ KeyVault URI & Secret
 - secret squirrel nut stash
 - ✓ CosmosDB & Container name

Virtual Joel



empty input, input
ie format.

```
parse_expenses.py ×  addresses.rb ×  
  
1  import datetime  
2  
3  def parse_expenses (expenses_string):  
4      """Parse the list of expenses and  
5      Ignore lines starting with #.  
6      Parse the date using datetime.  
7      Example expenses_string:  
8          2023-01-02 -34.01 USD  
9          2023-01-03 2.59 DKK  
10         2023-01-03 -2.72 EUR  
11      """  
12      expenses = []  
13  
14      for line in expenses_string.splitlines():  
15          if line.startswith("#"):  
16              continue  
17          date, value, currency = line.split()  
18          expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),  
19                          float(value),  
20                          currency))  
21      return expenses  
22  
23  expenses_data = '''2023-01-02 -34.01 USD  
24                    2023-01-03 2.59 DKK  
25                    2023-01-03 -2.72 EUR'''  
26  
27  
28  
29  
30  
31  
32  
33  
34
```


Let's fail fast, VS Code & CoPilot



TRADITIONAL APPROACH

- Microsoft Learn Modules: ~15 hours
- Quick Start Modules: 3-4 hours
- Stackoverflow
- W3Schools, etc
- Training: internal, external, Udemy, LinkedIn

Ask a question or type '/' for commands